

Adaptive Room Planner — MVP spec

DivHacks 2026 · companion to the PRD

Sep 26, 2026 · @Siddhi Mhatre

Pitch

Scan your room with your phone, text furniture ideas over iMessage, and rearrange your real room like a cozy top-down game before you buy or move anything.

Problem: people buy a desk from Facebook Marketplace, haul it home, and find it blocks the door or the window. Tape measures and floor-plan apps are tedious, and they forget what you already own and refuse to move.

Who it's for: renters in small NYC apartments, movers, anyone furnishing on a budget.

Hook for judges: the room is real (RoomPlan scan), the input is casual (iMessage), the editing feels like a game, and every answer is a named variant, so the Current Room is never lost.

User flow

Same six steps as the PRD; the words below are the build view of each.

1. Scan: the user walks the room with the iOS app. RoomPlan captures walls, doors, windows, dimensions, and detected objects.
2. Save the skeleton: the empty room (walls and openings only) is stored as its own record so every layout starts from the same shell. Never edited after the scan.
3. Build the current room: RoomPlan's detected objects seed the "Current Room" as sized boxes; the user swaps or adds owned furniture from the preset low-poly GLB library, then drags, rotates and locks. (Blender-from-photo is stretch, not in this step.)
4. Text Photon: the user sends an iMessage with a furniture photo, a listing link, or a request like "will this fit beside my window without moving my bed" or "make space for yoga".
5. Agent edits: the router imports any furniture, Gemini returns intent and zone-level actions with Backboard context, the placement solver turns zones into coordinates, and the result is saved as a named variant ("Marketplace Desk", "Yoga corner").

6. Play and compare: the user opens the web app, drags and rotates top-down, locks pieces, sees fit validation live, and flips or ghost-overlays variants against the Current Room.

MVP scope

Ship one room, end to end, with one iMessage request that visibly changes it and a compare view that proves it. Everything else is a slide.

Feature	MVP?	Why
RoomPlan scan to skeleton JSON (plus sample room and manual dimensions fallback)	In	Core. Apple does the hard part; the fallback keeps the demo alive without LiDAR
Empty-room baseline stored separately from layouts	In	PRD requirement: every variant builds on the same shell
Top-down Three.js room with add, drag, rotate, lock, remove	In	This is the demo people remember
Fit validation: bounds, overlap, door swing, 0.9 m door clearance, requested clear zones	In	PRD section 8; stops furniture clipping through walls; cheap with bounding boxes on a 10 cm grid
Furniture import from link, photo, or manual dims	In	The Marketplace desk has to arrive with real dimensions
Save and compare named variants, Current Room never overwritten	In	Shows the agent's work side by side; compare view is P0 in the PRD
Photon iMessage: link, photo, or request in; summary plus link out	In	The wow input
Gemini turns request into intent plus zone-level actions	In	Needed for "will this fit" and "make space for yoga"
Backboard preference memory	In, thin	Store must-keep items and one style or budget note; recall once in the demo
Pre-built low-poly GLB furniture library (15 to 20 items)	In	Built before judging so the demo never waits on Blender

Feature	MVP?	Why
Open floor % and walkable / keep-clear overlays	In, thin	One number and one overlay; the full analysis panel is P1
Blender MCP generating a new GLB from a photo	Stretch	Slow and flaky live; PRD lists perfect 3D from photos as out. Fall back to a sized, colored box
Video house tour + Apple Measure via Photon	Cut	RoomPlan already measures; two scan paths double the work
Multi-room / whole apartment	Cut	One room proves it
Photoreal textures of the user's actual room	Cut	Cozy low-poly looks better and is faster
ElevenLabs voice, WebXR, full AR preview	Cut	Out of MVP in the PRD; one roadmap line each

Tech stack

Layer	Tech	Job in the MVP
Scanner	Swift + RoomPlan	Export CapturedRoom as JSON (walls, doors, windows, objects, all in meters) and POST to the backend
Frontend	Three.js + TypeScript (Vite; React + react-three-fiber for the panels)	Orthographic top-down camera, drag/rotate/lock, fit validation, variant switcher and ghost compare
Backend	FastAPI	Rooms, furniture, layouts, agent pipeline, placement solver, Photon webhook
Database	MongoDB Atlas	Skeletons, furniture catalog, layouts and variants, agent request log
AI reasoning	Gemini API	Parse requests and listings into intent and structured actions; vision for photos

Layer	Tech	Job in the MVP
Memory	Backboard	Must-keep items, style, budget across chats
Messaging	Photon	iMessage in and out
3D assets	Blender + Blender MCP	Low-poly GLB library before the event; stretch: new GLB from a photo
Hosting	Vercel (frontend), Render or Railway (FastAPI)	Public demo URL

Data model

Five collections. Units are meters, rotation is degrees around the vertical axis, origin is the room's bottom-left corner, and floor position is `x, z` (Three.js floor plane). The PRD uses these same names.

Collection	Key fields	Notes
rooms	<code>_id, userId, name, skeleton { walls[], doors[] {wall, offset, width, swing}, windows[] {wall, offset, width, sillHeight}, floorPolygon }, source (scan / manual / sample), createdAt</code>	Skeleton is never edited after the scan
furniture	<code>_id, userId, name, category, dims {w, d, h}, glbUrl, source (preset / link / photo / manual), sourceUrl, price, color, estimated</code>	Catalog items plus items parsed from links and photos; <code>estimated</code> flags guessed dims
layouts	<code>_id, roomId, name, isCurrent, parentLayoutId, items [{furnitureId, x, z, rotation, locked}], zones [{label, x, z, w, d}], metrics {openFloor, conflicts}, createdBy (user / agent), requestText</code>	"Current Room" is the first layout with <code>isCurrent: true</code> ; variants fork from it

Collection	Key fields	Notes
users	<code>_id</code> , <code>phone</code> , <code>backboardUserId</code> , <code>createdAt</code>	Phone number is the Photon identity
agent_requests	<code>_id</code> , <code>userId</code> , <code>channel</code> , <code>rawText</code> , <code>mediaUrls</code> , <code>intent</code> , <code>actions[]</code> , <code>resultLayoutId</code> , <code>status</code>	Log for debugging and the demo replay

Sample layout item:

```
{ "furnitureId": "desk_01", "x": 1.2, "z": 3.4, "rotation": 90, "locked": false }
```

API endpoints

Method	Path	Does
POST	<code>/rooms</code>	iOS app uploads RoomPlan JSON (or the web app sends manual dims / sample); server stores the skeleton and creates the "Current Room" layout from detected objects
GET	<code>/rooms/{id}</code>	Skeleton plus list of layouts
GET	<code>/layouts/{id}</code>	One layout with resolved furniture (dims, GLB URL) and validation results
PUT	<code>/layouts/{id}</code>	Save drag, rotate, lock, add and remove changes from the web app
POST	<code>/layouts/{id}/fork</code>	Copy a layout under a new name
GET	<code>/layouts/{a}/compare/{b}</code>	Metric deltas and the list of items that moved
GET	<code>/furniture</code>	Catalog for the sidebar
POST	<code>/furniture/from-link</code>	Scrape the listing; Gemini extracts name, dims, price, color
POST	<code>/furniture/from-photo</code>	Gemini vision estimates category and dims from a photo, marked <code>estimated</code>

Method	Path	Does
POST	<code>/agent/request</code>	Run the agent pipeline on a layout; returns the plan and a new variant
POST	<code>/webhooks/photon</code>	Photon calls this on each iMessage; routes to the endpoints above and replies

Agent pipeline

Gemini decides what to do; plain Python decides exact coordinates; the browser's fit validation has the final say. LLMs are bad at geometry, so never trust raw x/z from the model.

1. Photon webhook (or the in-app request box) receives text, a link, or a photo.
2. Router classifies: new furniture (link or photo → `/furniture/from-*`), a layout request, or a question.
3. Backboard returns the user's saved context: must-keep items, style, budget.
4. Gemini gets the skeleton, the base layout with locks, furniture dims, and user context. It returns JSON only: `intent` (fit_item, make_space, keep_clear, compare, clarify), `constraints` (lock, adjacent, keep_clear, clear_zone), and `actions` (add, move, remove, rotate) with a target zone like "along north wall" or "near window".
5. Placement solver turns zones into coordinates and runs the fit rules: wall bounds, overlaps, door swing and 0.9 m clearance, a 0.75 m access edge on beds and desks, requested clear zones, and a walkable path from the door. Locked items are never moved. A failing plan is retried once with the violation in the prompt, then the old layout is kept.
6. Server saves the result as a new named layout (forked from the base) and texts back a one-line summary plus a link to the variant.
7. Backboard stores anything new the user revealed ("never move my bed", "I do yoga every morning").

Example Gemini output for the demo request:

```
{
  "intent": "fit_item",
  "variantName": "Marketplace Desk",
  "constraints": [
    { "type": "lock", "item": "bed_01" },
    { "type": "adjacent", "item": "desk_mkt", "feature": "window" }
  ],
  "actions": [
    { "type": "add", "item": "desk_mkt", "zone": "window wall, beside window" }
  ],
  "reply": "It fits on the window wall with 18 cm to spare; the bed didn't move."
}
```

Example for a lifestyle request:

```
{
  "intent": "make_space",
  "variantName": "Yoga corner",
  "constraints": [ { "type": "lock", "item": "dresser_01" }, { "type": "clear_zone", "w_m": 1.8, "d_m": 1.2, "label": "Yoga" } ],
  "actions": [ { "type": "move", "item": "desk_01", "zone": "east wall, centered" } ],
  "reply": "Moved the desk to the east wall to free 1.8 x 1.2 m by the window for your mat."
}
```

Frontend: cozy top-down

Think Unpacking or A Little to the Left, not a CAD tool.

- Camera: orthographic, tilted about 55 degrees, slight rotation so walls read as 3D. Scroll to zoom, right-drag to orbit in 90 degree steps.
- Look: low-poly GLBs, flat shading, soft warm light, one baked ambient-occlusion floor shadow per item. Walls cut to half height so you can see in.
- Drag: raycast to the floor plane, snap to a 10 cm grid, hold Shift to free move. Press R to rotate 90 degrees. Snap to walls.
- Fit feedback: item turns red on collision or in a keep-clear zone, small bounce when it lands, a soft "thunk" sound. Lock icon pins it. A conflict list names the items.

- Measurements: hovering an item shows its dims and the gap to the nearest wall.
- Overlays: open floor % in a corner tile; toggle for walkable (green) and keep-clear (red) floor overlays.
- Sidebar: catalog on the left, variant tabs on top ("Current Room", "Marketplace Desk", "Yoga corner"), a toggle to overlay two variants as ghosts, and a request box that hits `/agent/request` when Photon is not available.
- Libraries: three, GLTFLoader, raycasting from Three itself. Skip physics engines.

Build plan

Assumes 24 hours and 4 people. Agree on the layout JSON shape and the plan JSON shape in hour 0 so everyone can work against mock data.

Hours	iOS (A)	Frontend (B)	Backend (C)	AI + assets (D)
0 to 2	RoomPlan sample app running	Vite + Three.js scene, mock room from the sample JSON	FastAPI + Atlas connected, schemas for all five collections	Photon account, webhook to ngrok
2 to 8	Export JSON, POST to <code>/rooms</code>	Walls from skeleton, add from catalog, drag + snap	Room, layout, fork, furniture routes	Blender MCP: 15 to 20 GLBs
8 to 14	Scan 2 real rooms, save fixtures	Rotate, lock, fit validation colors and conflict list	Link scraper, photo endpoint, Gemini extraction	Gemini intent and action prompt, placement solver
14 to 20	Help frontend polish; manual-dims fallback screen	Variant tabs, ghost compare, open floor tile, request box	<code>/agent/request</code> end to end, compare endpoint	Backboard memory, iMessage replies
20 to 24	Record backup scan video	Deploy to Vercel	Deploy API	Rehearse demo script with the Marketplace Desk fixture

Demo script (3 minutes)

Matches the PRD demo scenario: the Marketplace Desk is the main beat, Yoga corner is the second request if time allows.

- 0:00 Hook: "I bought a desk on Marketplace that didn't fit beside my window." Show the photo.
- 0:20 Scan: 20 seconds of RoomPlan on a phone, live or pre-recorded. The room appears top-down in the browser as "Current Room".
- 0:50 Play: drag the desk, rotate the dresser, lock the bed. Show the red collision flash and the door-clearance zone.
- 1:20 iMessage: text Photon the Marketplace listing and ask "Will this fit beside my window without moving my bed?" The desk shows up with real dims and a "Marketplace Desk" variant opens; the bed didn't move.
- 1:50 Second request (optional): text "make space for yoga, keep my dresser." A "Yoga corner" variant appears.
- 2:20 Compare: ghost overlay of Current Room vs Marketplace Desk, with open floor % and what moved.
- 2:40 Close: memory recall ("You said never move the bed, so I didn't") and one line on future uses.

Risks and fallbacks

Risk	Fallback
RoomPlan needs a LiDAR iPhone (Pro models)	Borrow one early; keep 2 saved scans as JSON fixtures; sample room and manual dims in the web app
Venue Wi-Fi breaks live scan or iMessage	Pre-recorded clips of both; web app runs on cached data
Listing sites block scraping (Marketplace needs login)	Accept a screenshot instead; Gemini reads dims from the image
Listing has no dimensions	Gemini guesses from category, marks it <code>estimated</code> , user edits

Risk	Fallback
Gemini returns bad JSON or overlapping items	Schema validation and the solver reject, retry once with the violation, then keep the old layout
Blender MCP too slow live	Colored box with correct dims, swapped for a GLB later
Photon setup delay	Same endpoint callable from the request box in the web app

Future applications

- Real estate: agents scan a listing once, buyers text "would my king bed fit?" and get a staged variant.
- Architecture and renovation: the empty skeleton becomes the base for wall and layout proposals.
- Business and design proposals: designers send clients three named variants instead of mood boards.
- Moving: plan the new apartment before the truck arrives, using furniture you already own.
- Accessibility: check wheelchair turning radius (1.5 m) and clear paths automatically with the same solver.